Trasluz.

Liquidación B2B confidencial sobre Avalanche eERC

Confidencial para el mercado.
Transparente para el regulador.

Track Avalanche · Argentina Space 2026

● En vivo en Avalanche Fuji

EL PROBLEMA

Pagar a proveedores on-chain hoy obliga a elegir entre dos males

BLOCKCHAIN PÚBLICA

Todo a la vista

Tu competencia ve cuánto le pagás a cada proveedor y reconstruye tus precios, tus márgenes y tu flujo de caja.

PRIVACIDAD TOTAL

Auditoría imposible

Si nadie puede ver los montos, el regulador tampoco. Ningún banco ni empresa regulada puede adoptar un riel así.

> Una empresa necesita **las dos cosas a la vez**: que el mercado no vea sus montos y que su auditor sí.

LA SOLUCIÓN

Pago contra entrega con montos cifrados y una ventana para el auditor

ESCROW

Contra entrega

Un vault por obligación. El pago se libera sólo cuando el proveedor confirma la entrega.

CONFIDENCIAL

Monto cifrado

eERC oficial de Ava Labs: ElGamal y pruebas Groth16 generadas en el navegador.

AUDITABLE

Viewing key

El auditor designado descifra cada operación y concilia lo fondeado contra lo liberado.

KYC

Riel habilitado

Sólo las entidades de la allowlist pueden crear y cobrar obligaciones.

> El diferenciador del eERC no es la privacidad: es la **auditabilidad selectiva**.

CÓMO FUNCIONA

Cinco pasos, cero montos en claro on-chain

1

EMPRESA

Crea

La obligación nace con un vault de escrow propio, registrado en eERC.

2

EMPRESA

Fondea

Transferencia confidencial al vault, con una prueba ZK.

3

PROVEEDOR

Confirma

Descifra el saldo del vault, verifica el monto y confirma la entrega.

4

EMPRESA

Libera

El vault paga al proveedor. El destino lo fija el contrato.

5

AUDITOR

Concilia

Con su viewing key ve los montos reales: fondeado = liberado.

Estados: **Creada** → **Fondeada** → **Liberada** Sin entrega confirmada: **Cancelada** → reembolso al pagador

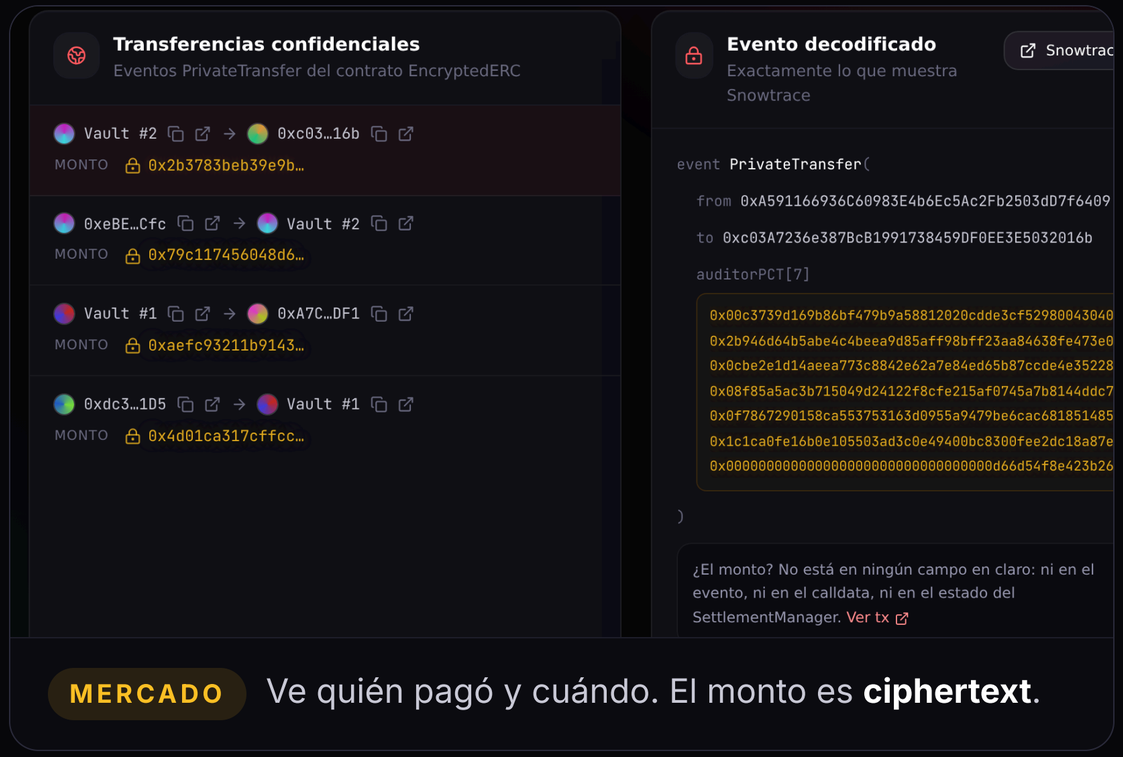
QUIÉN VE QUÉ

El mismo dato, tres miradas

DATO	MERCADO / SNOWTRACE	PROVEEDOR	AUDITOR DESIGNADO
Pagador, proveedor, estado y fechas	visible	visible	visible
Monto de cada pago	**ciphertext**	el suyo	**descifrado**
Saldos y flujo de caja	**ciphertext**	el suyo	**conciliable**
Factura	sólo un hash	hash	hash
Depósito y retiro de USDC (rampa)	visible	visible	visible

DEMO EN FUJI

La misma transferencia, vista al trasluz





ARQUITECTURA

Componemos sobre el estándar: no tocamos la criptografía

Frontend React + wagmi
Empresa · Proveedor · Auditor · Vista pública. Pruebas Groth16 en un Web Worker.

↓

SettlementManager.sol
Allowlist KYC, estados, CEI + nonReentrant. Eventos y reverts sin montos.

EscrowVault
Un clon EIP-1167 por obligación. Sólo obedece al manager.

↓ transfer confidencial

EncryptedERC oficial · converter mode
Sin modificar: Registrar, verifiers Groth16 de producción y auditor designado.

↓

Mock USDC en Avalanche Fuji

DECISIÓN CLAVE

Un vault eERC por obligación

En eERC sólo mueve fondos quien conoce la clave del emisor. Cada vault tiene su propia clave, pero **esa clave no autoriza gastos**: el destino lo fija el contrato.

PORTAL AUDITOR

100 % local: descifra cada `PrivateTransfer`, concilia y exporta CSV.

DESPLEGADO

En vivo en Avalanche Fuji, verificable en Snowtrace

SettlementManager	0xDAe1bA1e0FDFb5c34bB894120eBe8Cd114e05592
EncryptedERC	0x8391B1756A7fe2aCE4246aBe1b9cB5Eb08607713
Registrar	0x0564C86206A20D721D54FDdC12528E462769D822
Mock USDC	0xE8adcE9391B2F06517738e654b32DBbCb07E30A6
Auditor designado	0x8C6861bb10dfF6fD2eC115ddF5dE369aCC68a185

11
contratos desplegados

2/2
obligaciones conciliadas

22
tests con pruebas ZK reales

App en vivo: `trasluz-2av.pages.dev`

Obligación #2 · 18.750,25 USDC · transacciones:

- Depósito `0x737f…45ba`
- Crear `0x79b4…67b9`
- Fondeo `0x2d9e…10ef`
- Entrega `0x3195…bca1`
- Liberación `0xdb08…b932`

SEGURIDAD

Pensado para que un equipo de compliance lo pueda aprobar

Sin cripto propia

eERC oficial copiado byte a byte, con verifiers y zkeys de la ceremonia de Ava Labs.

Sin fugas de monto

Ninguna función recibe montos. Los tests escanean eventos y calldata.

Escrow sin desvío

El destino lo fija el contrato y eERC valida la clave del receptor en la prueba.

Sin doble liberación

Checks-effects-interactions y nonReentrant. El vault queda en cero.

Sin reembolso tras la entrega

El pagador sólo recupera fondos de una obligación cancelada.

Viewing key en runtime

Se carga en el portal, vive en memoria y se valida contra el contrato.

Limitaciones declaradas: la allowlist KYC es manual · depósitos y retiros de USDC son públicos (es la rampa ERC-20) · un pago parcial no se puede impedir on-chain: el auditor lo marca como remanente.

LO QUE SIGUE

Roadmap

- **L1 permisionada** de Avalanche: la allowlist gatea también el token, no sólo el riel.
- **Safe multisig + timelock** como owner de la allowlist.
- **KYC real** con un proveedor de identidad.
- **Plazos con auto-reembolso** y liberación firmada por el proveedor.
- **Mainnet C-Chain** con USDC nativo.



Confidencial para el mercado. Transparente para el regulador.

Demo: `trasluz-2av.pages.dev`

Código: `github.com/Maurezg7/trasluz`